

4. WAP to implement singly linked list with the following operations
- Create a linked list
 - Insertion of a node at → first position, any position, end of the list
 - Display the contents of linked list.

PSEUDO CODE

```
struct node {
```

```
    int data;
```

```
    struct node *next;
```

```
};
```

```
void insertAtBeginning (int data) {
```

```
    struct node*
```

```
    struct node * head = NULL;
```

```
    newNode = (struct node*) malloc (size of (struct node));
```

```
    newNode → next = NULL;
```

```
    if (head == NULL) {
```

```
        head = newNode = temp;
```

```
    }
```

```
    else {
```

```
        temp → next = newNode;
```

```
        temp = newNode;
```

```
    }
```

```
void insertAtBeginning (int data) {
```

```
    struct node* newNode = (struct node*) malloc (size of (struct node));
```

```
    newNode → data = data;
```

```
    newNode → next = head;
```

```
    head = newNode;
```

```
}
```

```

void InsertAtEnd (int data){
    struct node* newNode = (struct node*) malloc (sizeof (struct node));
    newNode -> data = data;
    newNode -> next = NULL;
    if (head == NULL){
        head = newNode;
    }
    else {
        struct node* temp = head;
        while (temp -> next != NULL){
            temp = temp -> next;
        }
        temp -> next = newNode;
    }
}

```

```

void InsertAtAnyPosition (int data, int pos) {
    newNode -> data = data;
    for (int i = 1; i < pos - 1; i++) {
        temp = temp -> next;
    }
    if (temp == NULL) {
        free (newNode);
    }
    else {
        newNode -> next = temp -> next;
        temp -> next = newNode;
    }
}

```

```

void display() {
    while (temp != NULL) {
        printf ("%d", temp -> data);
        temp = temp -> next;
    }
}

```

10/11/25

CODE:

```

#include <stdio.h>
#include <stdlib.h>

struct node {
    int data;
    struct node *next;
};

struct node *head = NULL;

void createlist (int n) {
    struct node *newnode, *temp;
    int data, i;
    if (n <= 0) {
        printf("n should be greater than 0.");
        return;
    }
    for (i = 1; i <= n; i++) {
        newnode = (struct node *) malloc (sizeof (struct node));
        if (newnode == NULL) {
            printf("Memory allocation failed.");
            return;
        }
        printf("Enter data: ");
        scanf ("%d", &data);
        newnode->data = data;
        newnode->next = NULL;
        if (head == NULL) {
            head = newnode;
        }
        else {
            temp->next = newnode;
        }
        temp = newnode;
    }
}

```

```

}
printf("Linked list is created.");
}

void insertAtBeginning (int data) {
    struct node *newnode = (struct node*) malloc (size of (struct node));
    newnode -> data = data;
    newnode -> next = head;
    head = newnode;
    printf("Node inserted at the beginning.");
}

void insertAtEnd (int data) {
    struct node *newnode = (struct node*) malloc (size of (struct node));
    newnode -> data = data;
    newnode -> next = NULL;
    if (head == NULL) {
        head = newnode;
    }
    else {
        struct node *temp = head;
        while (temp -> next != NULL) {
            temp = temp -> next;
        }
        temp -> next = newnode;
    }
    printf("Node inserted at the end.");
}

void insertAtPosition (int data, int pos) {
    int i;
    struct node *newnode, *temp = head;
    if (pos < 1) {
        printf("Invalid position.");
        return;
    }
}

```

```

if (pos == 1) {
    insertAtBeginning(data);
    return;
}

newnode = (struct node*) malloc (Size of (struct node));
newnode -> data = data;

for (i=1; i < pos-1; i++) {
    temp = temp -> next;
}

if (temp == NULL) {
    printf("Position out of range.\n");
    free(newnode);
} else {
    newnode -> next = temp -> next;
    temp -> next = newnode;
    printf("Node inserted at position %d", pos);
}
}
}

```

```

void display() {
    struct node *temp = head;
    if (head == NULL) {
        printf("List is empty.\n");
    } else {
        printf("Linked List: \n");
        while (temp != NULL) {
            printf("%d -> ", temp -> data);
            temp = temp -> next;
        }
    }
}

```

```
int main() {
```

```
    int ch, n, data, pos;
```

```
    do {
```

```
        printf("\n 1) Create linked list 2) Insert at beginning
```

```
        3) Insert at any position 4) Insert at end
```

```
        5) Display list");
```

```
        printf("\n Enter your choice:");
```

```
        scanf("%d", &ch);
```

```
        switch (ch) {
```

```
            case 1:
```

```
                printf("Enter no. of nodes:");
```

```
                scanf("%d", &n);
```

```
                createList(n);
```

```
                break;
```

```
            case 2:
```

```
                printf("Enter data:");
```

```
                scanf("%d", &data);
```

```
                insertAtBeginning(data);
```

```
                break;
```

```
            case 3:
```

```
                printf("Enter data, position:");
```

```
                scanf("%d %d", &data, &pos);
```

```
                insertAtPosition(data, pos);
```

```
                break;
```

```
            case 4:
```

```
                printf("Enter data:");
```

```
                scanf("%d", &data);
```

```
                insertAtEnd(data);
```

```
                break;
```

```
            case 5:
```

```
                display();
```

```
                break;
```

```
            default: printf("Invalid input.");
```

}

Output:

1) Create linked list 2) Insert at beginning 3) Insert at any position 4) Insert at end 5) Display list

Enter your choice: 1

Enter no. of nodes: 3

Enter data: 10

Enter data: 20

Enter data: 30

Linked list is created:

1) Create linked list 2) Insert at beginning 3) Insert at any position 4) Insert at end 5) Display list

Enter your choice: 2

Enter data: 5

Node inserted at the beginning.

1) Create linked list 2) Insert at beginning 3) Insert at any position 4) Insert at end 5) Display list

Enter your choice: 4

Enter data: 40

Node inserted at the end.

1) Create linked list 2) Insert at beginning 3) Insert at any position 4) Insert at end 5) Display list

Enter your choice: 3

Enter data, position: 15 3

Node inserted at position 3

1) Create linked list 2) Insert at beginning 3) Insert at any position 4) Insert at end 5) Display list

Enter your choice : 5

~~Linked list :~~

5 → 10 → 15 → 20 → 30 → 40 →

17/11/25
Seen